

CPSC 340: Machine Learning and Data Mining

Autoencoders and Multi-label Classification

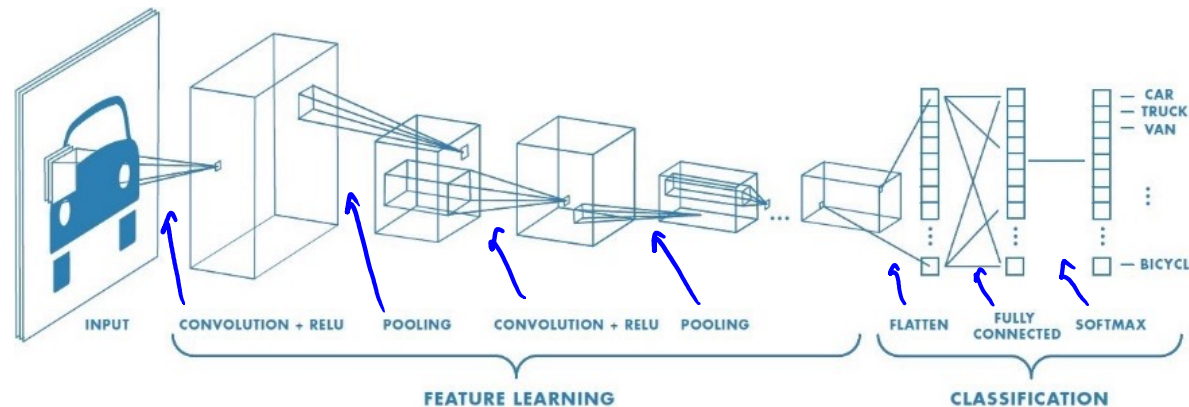
Admin

- HW6 due on Wed Dec 3.
- Please bring a device to fill the SEI survey on Mon.

Last Time: Convolutional Neural Networks

- We discussed **convolutional neural network**:
 - Neural networks where layers perform several convolutions.
 - Drastically reduces number of parameters and computation time.
 - Gains a degree of translation invariance (“object can appear anywhere”).

general overview

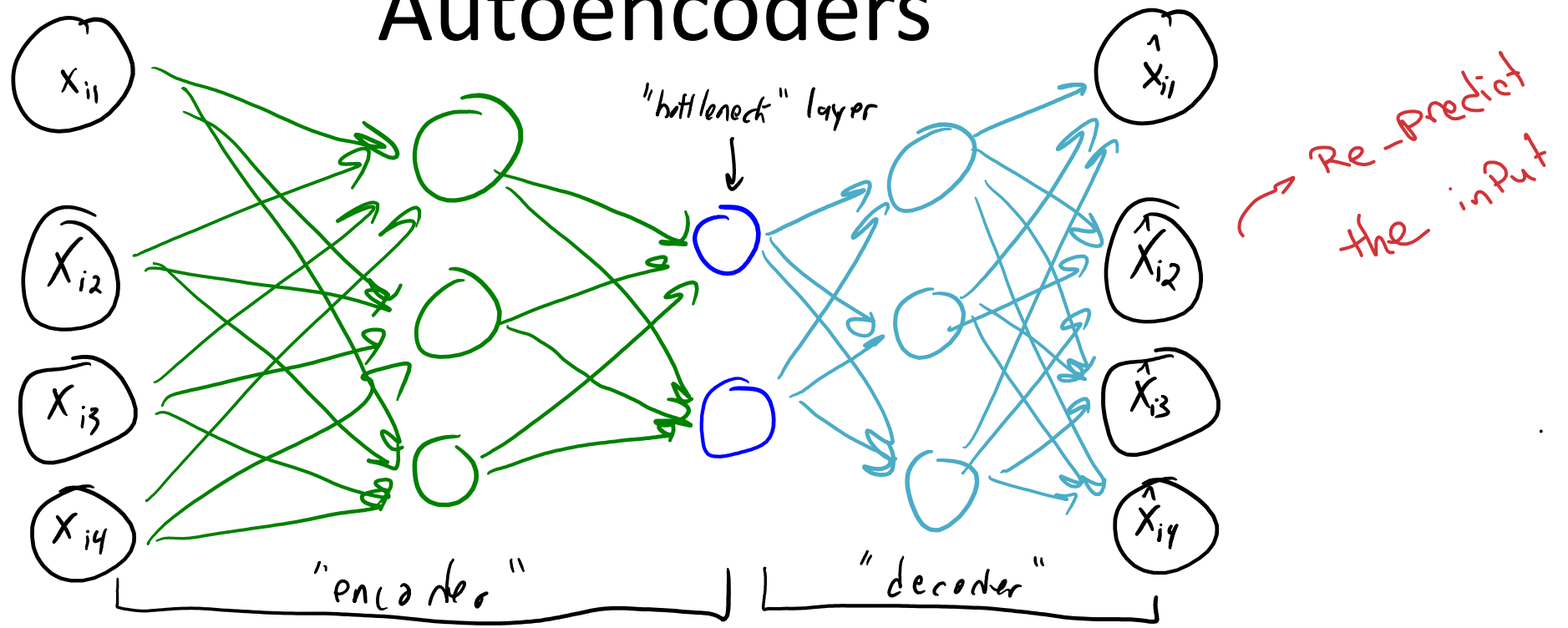


- **ImageNet**: Millions of labeled images, 1000 object classes.
 - Led to popularization of CNNs and deep learning across computer vision.
 - Led to many insights about how to train CNNs and construct architectures.
 - ImageNet + CNNs is arguably most influential computer vision work of all time.

Next Topic: Autoencoders

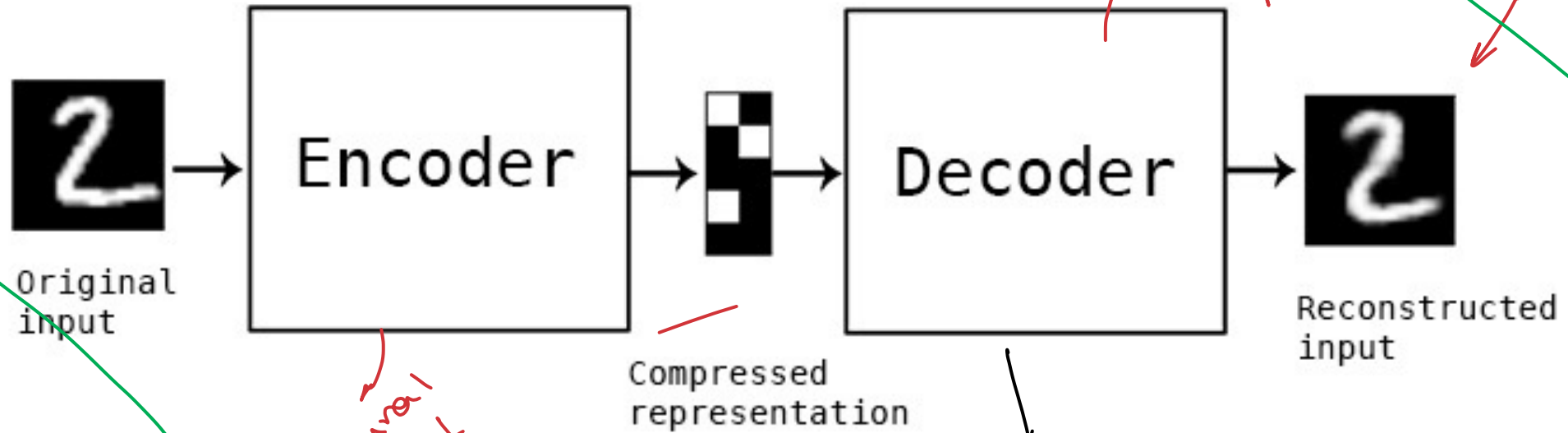
Special Types of NN

Autoencoders



- Autoencoders are neural networks with same input and output.
 - Includes a bottleneck layer: with dimension 'k' smaller than input 'd'.
 - First layers "encode" the input into bottleneck.
 - Last layers "decode" the bottleneck into a (hopefully valid) input.

Autoencoders



- This is an unsupervised learning method.

- There are no labels y_i .

- Relationship to principal component analysis (PCA):

- With squared error and linear network (no non-linear 'h'), equivalent to PCA.

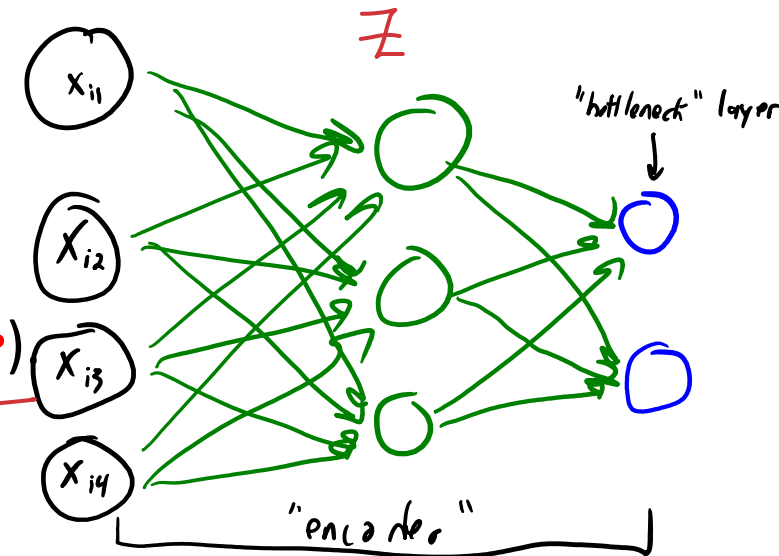
- Size of bottleneck layer gives number of latent factors 'k' in PCA.

- With non-linear transforms: a non-linear/deep generalization of PCA.

if they are linear
(Encoder and Decoder)
⇒ it would be PCA

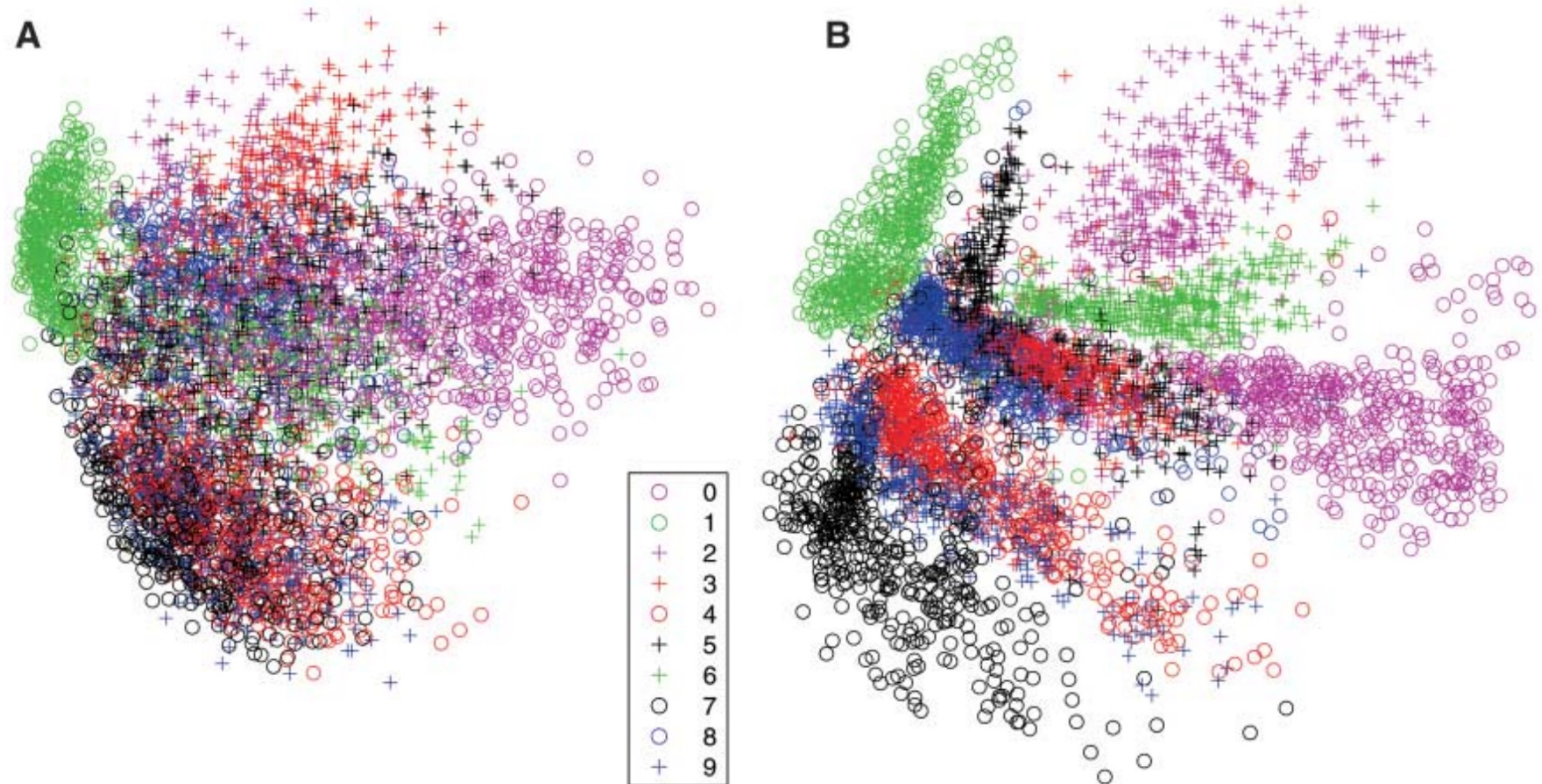
Encoder as Learning a Representation

- Consider the **encoder** part of the network:
 - Takes features ' x_i ' and makes low-dimensional ' z_i ' (*latent code*)
- Can use encoded z_i for the **usual latent-factor tasks**:
 - Use z_i as compressed input (reduce memory needed)
 - Set bottleneck **size to 2**, and plot z_i to visualize data ↘ visualize
 - Try to interpret what the bottleneck features z_i mean.
 - Use z_i for supervised learning: **add a supervised ' y_i '** to final layer of trained autoencoder, fit with SGD.
 - Called "**unsupervised pre-training**".
 - If you use unlabeled data to do this initialization, an example of "**self-supervised**" learning.
 - It's often **easier to get a lot of unlabeled data**

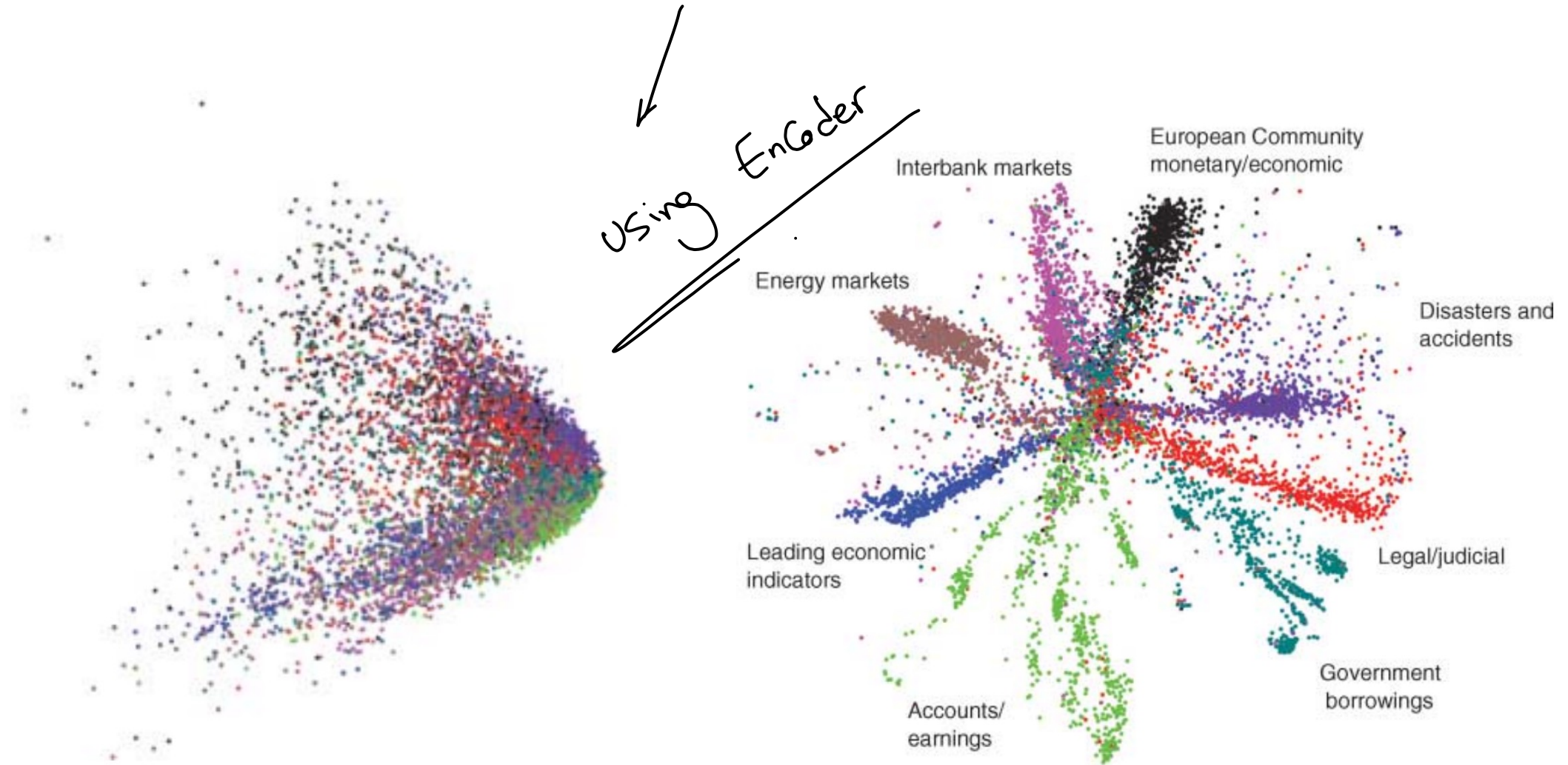


PCA vs. Deep Autoencoder (Digits Data)

Fig. 3. (A) The two-dimensional codes for 500 digits of each class produced by taking the first two principal components of all 60,000 training images. (B) The two-dimensional codes found by a 784-1000-500-250-2 autoencoder. For an alternative visualization, see (8).

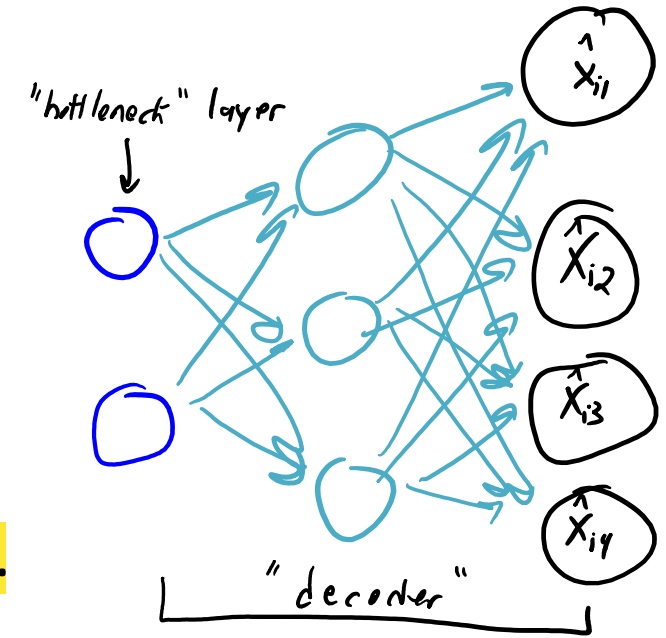


PCA vs. Deep Autoencoder (Document Data)



Decoder as Generative Model

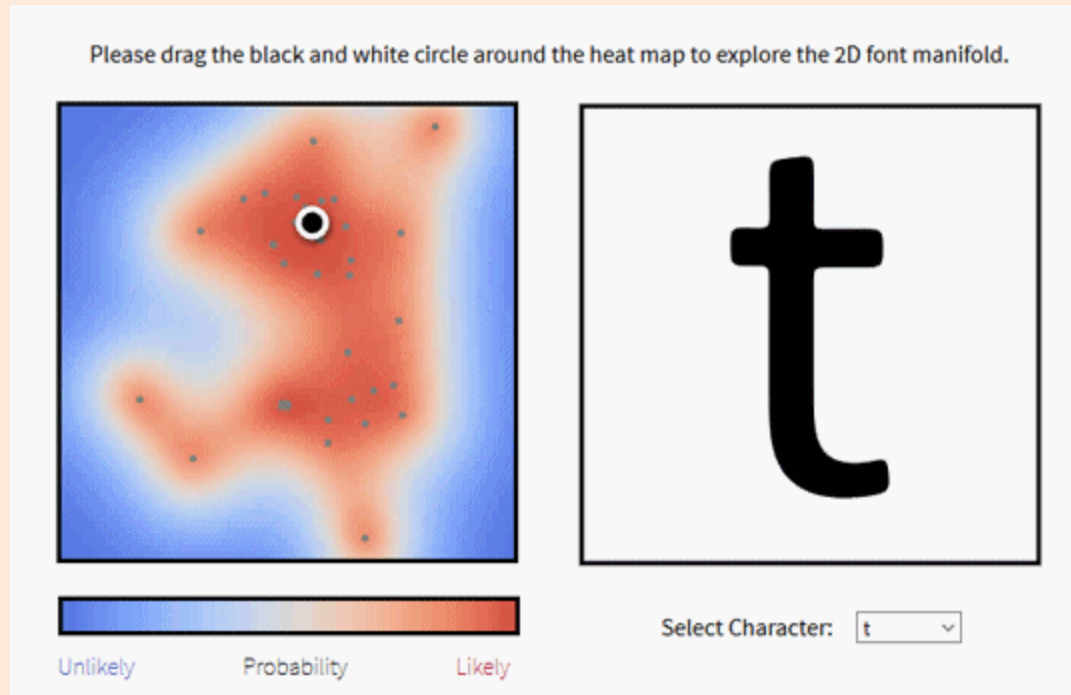
- Consider the **decoder** part of the network:
 - Takes low-dimensional ' z_i ' and makes features ' $\hat{x}_i$ '
- Can be used for **outlier detection**:
 - Check distance to original features to detect outliers.
- Can be used to generate new data:
 - The ' z_i ' close to training data might generate new reasonable x_i values.
 - Like in the Variational Autoencoders



if data point is far from
the decoder output → Outlier
detection

Font Manifold

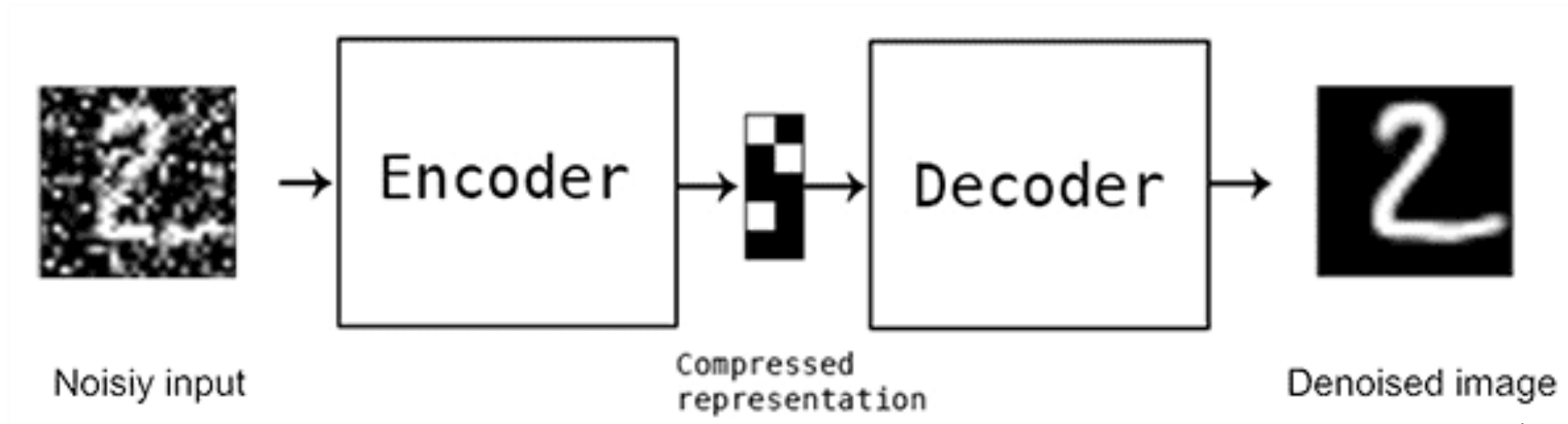
- Going from z_i to $\hat{x}_i$ for different fonts:



- The above was not actually generated by an autoencoder.
- But the decoder part of autoencoders is trying to do something like this.

Denoising Autoencoders

- A common variation on autoencoders is **denoising autoencoders**:
 - Use “**corrupted**” inputs, and learn to **reconstruct uncorrupted** originals.



- “Learn a model that **removes the noise**”.
- Often easy to get lots of training data, just add noise to “clean” data.
- You can apply the model to denoise new images.
- Does not necessarily need a “bottleneck” layer.

Noising is more difficult than denoising.

Image Colourization



Colorado National Park, 1941



Textile Mill, June 1937



Berry Field, June 1909



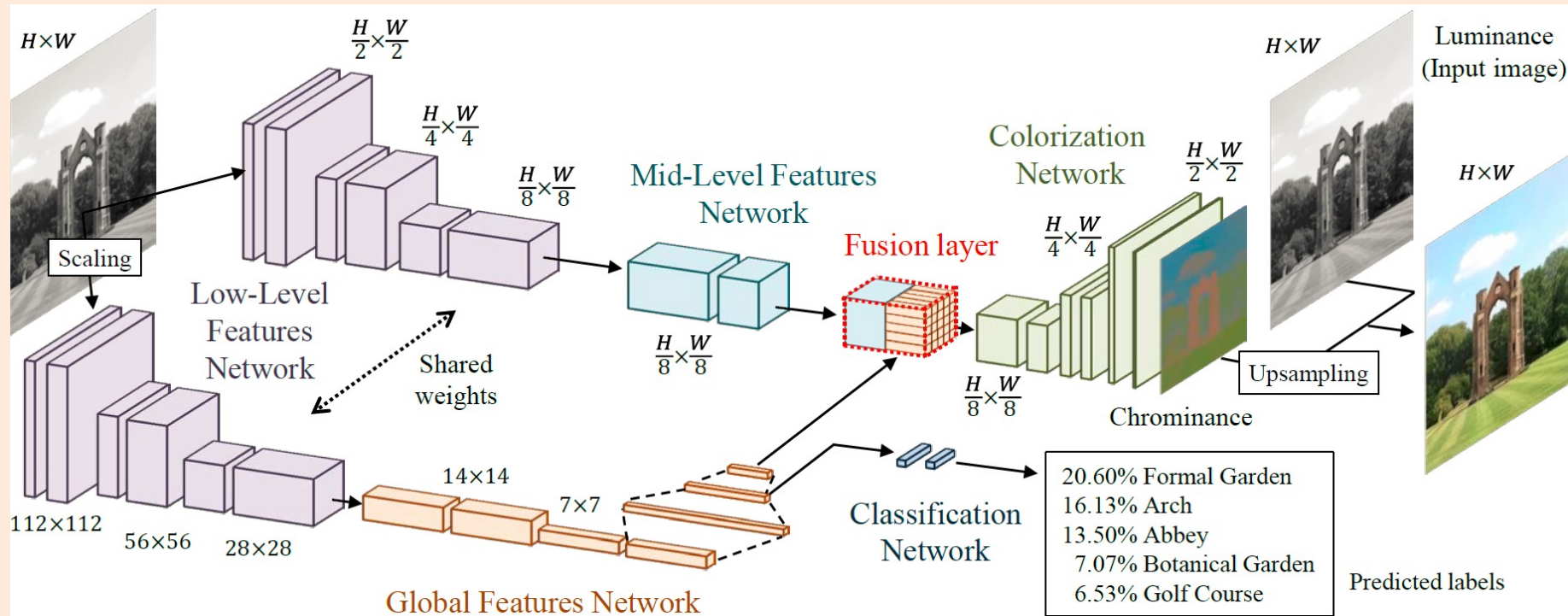
Hamilton, 1936



- Gallery: <http://iizuka.cs.tsukuba.ac.jp/projects/colorization/extra.html>
- Video: <https://www.youtube-nocookie.com/embed/ys5nMO4Q0iY>

Image Colourization

- Instead of noisy inputs, you use de-coloured inputs:



- Another application is **super-resolution**:
 - Learn to output a high-resolution image based on low-resolution images.

Zooming Pictures

Next Topic: Multi-Label Classification

Neural Networks with Multiple Outputs

- Previous neural networks we have seen **only have 1 output** ' y_i '.
- In autoencoders, **we have 'd' outputs** (one for each feature).

$$\begin{aligned}\hat{x}_{i1} &= v_1^T h(w^3 h(w^2 h(w^1 x_i))) \\ \hat{x}_{i2} &= v_2^T h(w^3 h(w^2 h(w^1 x_i))) \\ &\vdots \\ \hat{x}_{id} &= v_d^T h(w^3 h(w^2 h(w^1 x_i)))\end{aligned} \quad \left. \vphantom{\begin{aligned}\hat{x}_{i1} \\ \hat{x}_{i2} \\ \vdots \\ \hat{x}_{id}\end{aligned}} \right\} \hat{x}_i = V h(w^3 h(w^2 h(w^1 x_i)))$$

- For training, we **add up the loss** across all 'j':

$$f(w^1, w^2, v) = \sum_{i=1}^n \sum_{j=1}^d (\hat{x}_{ij} - x_{ij})^2$$

For AutoEncoder (red arrow pointing to the equation)

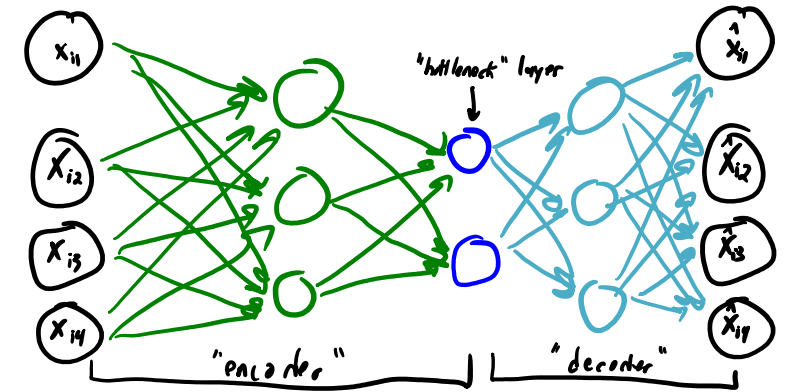
For example (red arrow pointing to the equation)

squared error for continuous x_{ij} (green bracket under the equation)

$= v_j^T h(w^3 h(w^2 h(w^1 x_i)))$ (green arrow pointing to the equation)

$$f(w^1, w^2, v) = \sum_{i=1}^n \sum_{j=1}^d \log(1 + \exp(-\hat{x}_{ij} x_{ij}))$$

logistic loss for binary $x_{ij} \in \{-1, +1\}$ (green bracket under the equation)

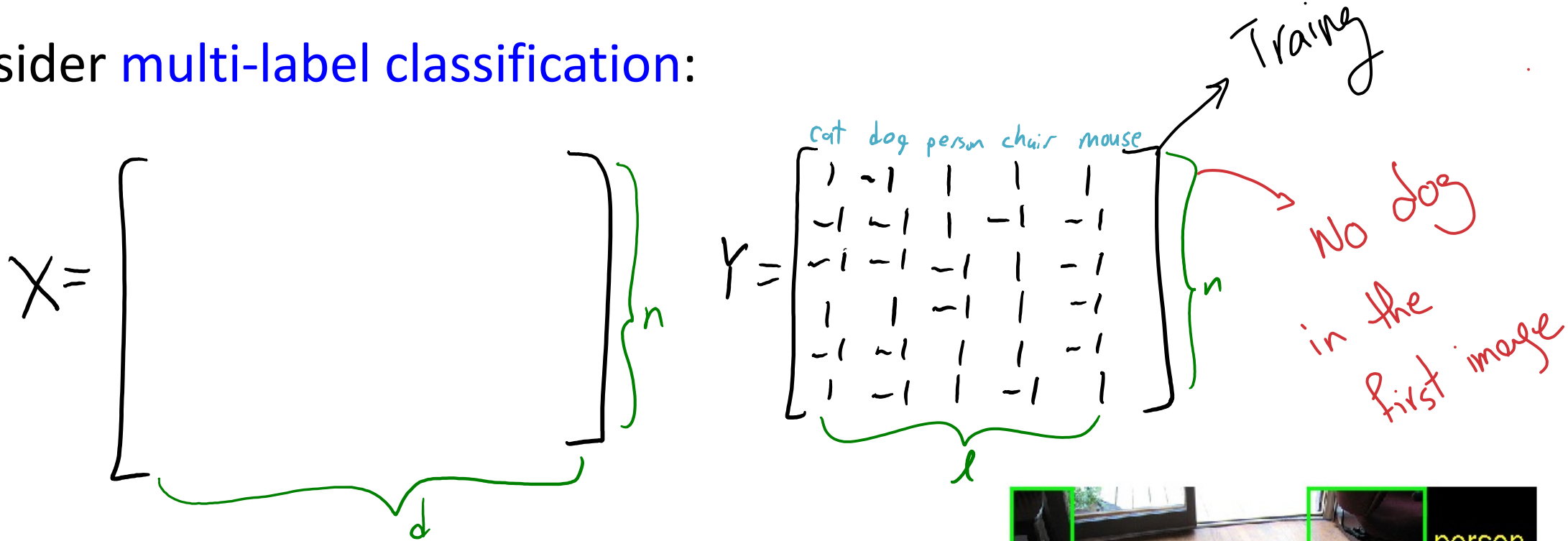


- Fit with SGD (sampling random 'i'), and usual deep learning tricks can be used.
 - Even though network has multiple outputs, '**f**' is a **scalar** so AD works as before.
 - For images, may want to **use convolution layers**.

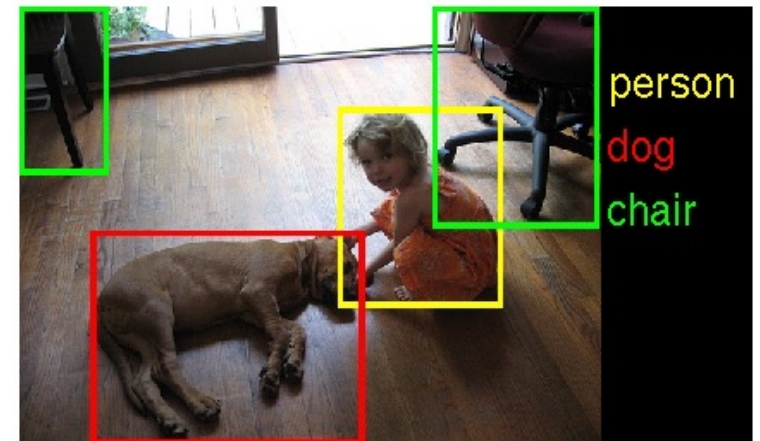
softmax (red arrow pointing to the equation)

Motivation: Multi-Label Classification

- Consider **multi-label classification**:



- Which of the ' ℓ ' objects are in this image?
 - There may be **more than one "correct" class label**.



Independent Classifier Approach

- One way to build a multi-label classifier:

- Train a classifier for each label.

- Train a neural network that predicts +1 if the image contains a dog, and -1 otherwise.
 - Train a neural network that predicts +1 if the image contains a cat, and -1 otherwise.
 - ...

- To make predictions for the ' ℓ ' classes.

- Apply all each label's binary classifier.
 - Predict all the resulting +1 values as the set of labels.

- Drawbacks:

- Lots of parameters: $\ell \times$ (number of parameters for base classifier).

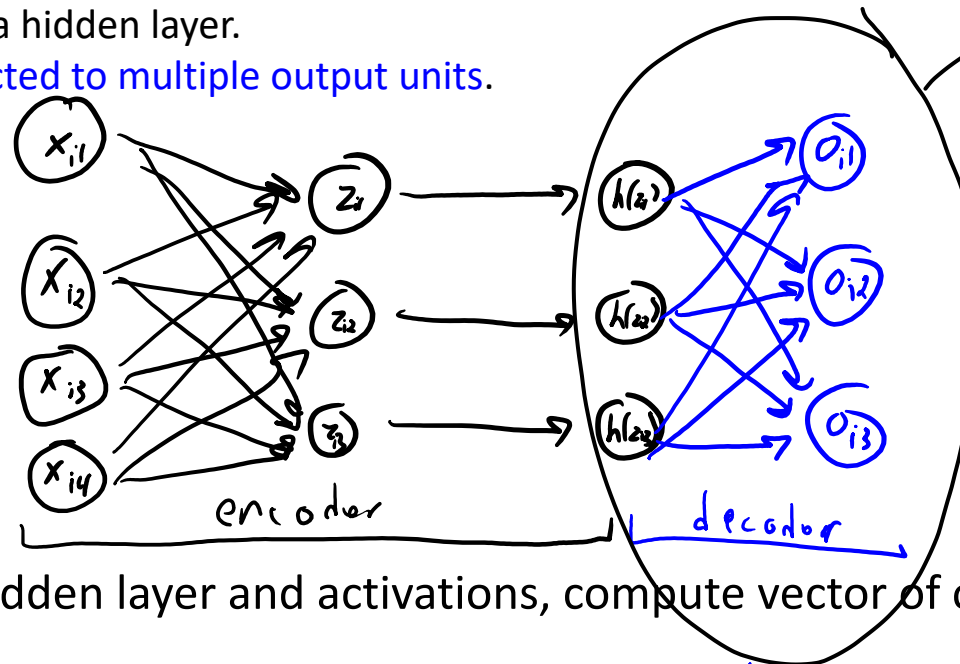
- Each classifier needs to “relearn from scratch”.

- Each classifier needs to learn its own Gabor filters, how corners and light works, and so on.
 - A lot of visual features for “dog” might also help us predict “cat”.

if it is
in Pixel
level
Segmentation

Encoding-Decoding for Multi-Label Classification

- Multi-label classification with an **encoding-decoding** approach:
 - Input is connected to a hidden layer.
 - Hidden layer is connected to multiple output units.



you only
separate
decoder
part of
model //

$$o_{i1} = v_1^T h(Wx_i)$$

$$o_{i2} = v_2^T h(Wx_i)$$

$$o_{i3} = v_3^T h(Wx_i)$$

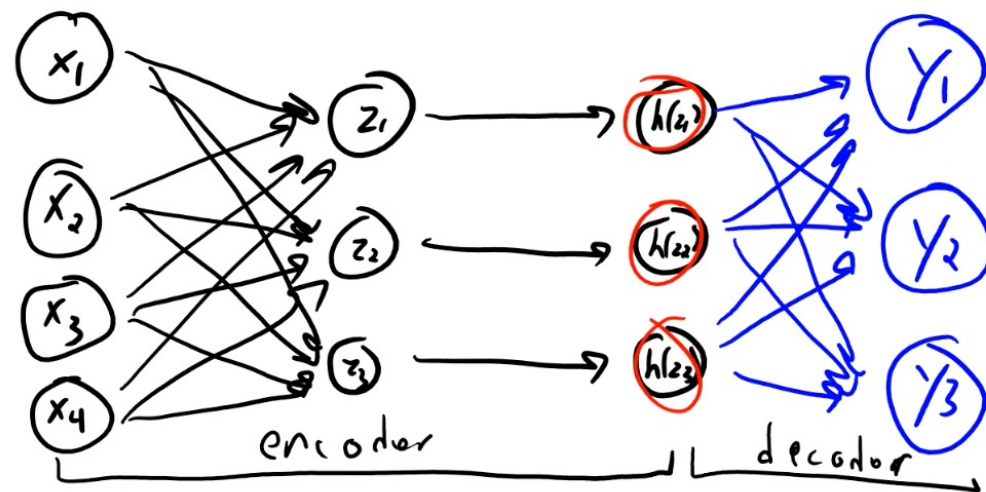
- Prediction: compute hidden layer and activations, compute vector of outputs, take **sign element-wise**:

$$\text{sign}(V h(Wx_i))$$

$l \times k$ $k \times d \times 1$

- Number of parameters and cost is $O(dk + k\ell)$ for ' ℓ ' classes and ' k ' hidden units.
 - If we trained a separate network for each class, number of parameters and cost would be $O(dk\ell)$ (' W ' for each class).
- Might have **multiple layers**, **convolution layers**, and so on. And no need to have a "bottleneck" layer.

Encoding-Decoding for Multi-Label Classification



- We usually assume that the **classes are independent given last layer:**

$$p(\underline{y_1}, \underline{y_2}, \dots, \underline{y_k} \mid \underline{x_1}, \underline{x_2}, \dots, \underline{x_d}, \underline{W}, \underline{V}) = p(\underline{y_1} \mid \underline{x_1}, \underline{x_2}, \dots, \underline{x_d}, \underline{W}, \underline{V}) p(\underline{y_2} \mid \underline{x_1}, \underline{x_2}, \dots, \underline{x_d}, \underline{W}, \underline{V}) \dots p(\underline{y_k} \mid \underline{x_1}, \underline{x_2}, \dots, \underline{x_d}, \underline{W}, \underline{V})$$

with: $p(\underline{y_1} = 1 \mid \underline{x}, \underline{W}, \underline{V}) = \frac{1}{1 + \exp(-\underline{v_1}^T \underline{h}(\underline{W}\underline{x}))}$ $p(\underline{y_2} = 1 \mid \underline{x}, \underline{W}, \underline{V}) = \frac{1}{1 + \exp(-\underline{v_2}^T \underline{h}(\underline{W}\underline{x}))}$...

Logistic loss (pointing to the first equation)

θ_1 (pointing to $\underline{v_1}$)

θ_2 (pointing to $\underline{v_2}$)

- This probabilistic model is ultimately a fancy **product of Bernoullis** model:

- Conditioned on the features and parameters, we have $p(\underline{y_1}, \underline{y_2}, \dots, \underline{y_k}) = p(\underline{y_1})p(\underline{y_2}) \dots p(\underline{y_k})$, where $p(\underline{y_c} = 1) = \theta_c$.
- This makes decoding and other inference problems easy: you do **inference on each $\underline{y_c}$ independently.**

Encoding-Decoding for Multi-Label Classification

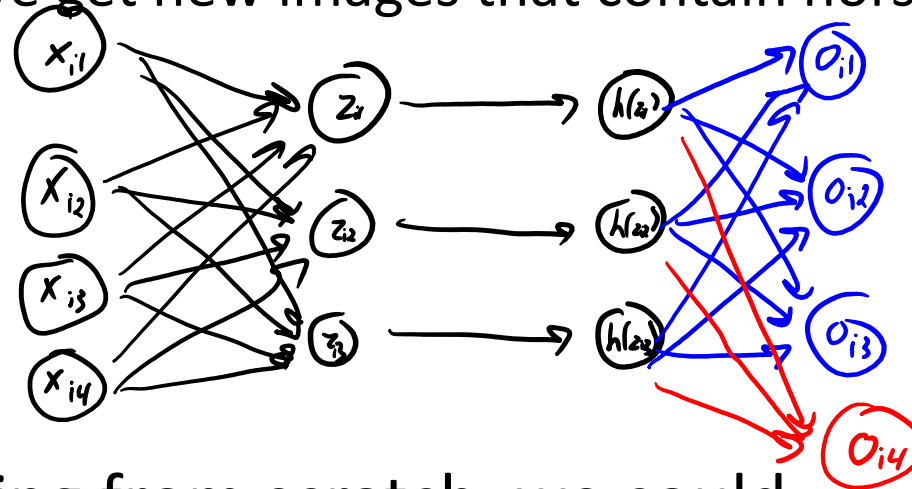
- Using sigmoid likelihood, **negative log-likelihood** we optimize for MLE:

$$f(W, V) = \sum_{i=1}^n \sum_{c=1}^{\ell} \log(1 + \exp(-y_c^i v_c^T h(Wx^i)))$$

- Use backpropagation or AD to compute gradient, train by SGD.
 - You randomly sample a training example ‘i’ and compute gradient for all labels.
 - The updates of ‘W’ lead to **features that are useful across classes**.
 - The updates of ‘V’ focus on getting the **specific class labels** right.
- Important:
 - Above we are assuming **independence of labels** given the last layer.
 - But the **last layer can reflect dependencies**.
 - If “dog” and “human” are frequently together, this should be **reflected in the hidden layer**.
 - For example, o_{ic} for “human” might be higher when we have a high o_{ic} value for “dog”.

Pre-Training for Multi-Label Classification

- Consider a scenario where we get a **new class label**.
 - For example, we get new images that contain horses (not seen in training).



- Instead of training from scratch, we could:
 - Add an **extra set of weights $v_{\ell+1}$** to the final layer for the new class.
 - Train these weights with **the encoding weights 'W' fixed**.
 - This is a simple/convex/easy logistic regression problem.
 - If we already have “features” that are good for many classes, we may be able to **learn a new class with very-few training examples!**

just need to
add it to the
last part.

Logistic Regression
based on
Encoder feature

Pre-Training for Multi-Label Classification

- Using an existing network for new problems is called “pre-training”
 - Typically, we start with a network trained on a large dataset.
 - We use this network to give us features to fit a smaller dataset.
 - “Few-shot learning”.
 - Depending the setup, you may also update ‘W’ and the other ‘V_c’.
 - Useful if you have a lot of data on the new class.
 - In this case, would typically mix in new examples with old ones.
 - Increasing trend in vision and language to using pre-training a lot.
 - No need to learn everything about vision/language for every task!
- pre-learning → get the part that matters based on previous data*

Summary

- **Autoencoders:**
 - Neural network where the output is the input.
 - Non-linear generalization of PCA.
 - **Encode** data into a bottleneck layer, then **decode** predict original input.
 - Can be used for visualization, compression, outlier detection, pre-training.
- **Denoising autoencoders** train to uncorrupt/enhance images.
 - Useful for removing noise, adding colour, super-resolution, and so on.
- **Encoding-Decoding** approach to multi-label classification:
 - Have all classes shared the same hidden layer(s).
 - Reduces number of parameters.
 - Models dependencies between classes, while keeping inference easy.
- **Pre-training:**
 - Use parameters from model trained on large diverse dataset, to initialize SGD for new dataset.